Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА №1

Изучение методов защиты программного обеспечения

по дисциплине
«Технологии реверс-инжиниринга программного обеспечения»

Выполнил
студент гр. 5151001/40202 И.А. Эков
Руководитель
ассистент ВШК ИКНК Е.Д. Кондачков

Санкт-Петербург – 2026

1 ЦЕЛЬ И ПОСТАНОВКА ЗАДАЧИ

Цель работы — изучить методы защиты программного обеспечения от анализа и изменения, реализовать их в программе crackme и исследовать способы обхода.

Программа считывает пароль из password.txt. При правильном пароле она формирует serial.txt с номером вида KEY$xxxxxxxxxx$ и запускает полезные вычисления. При неверном пароле исходная защищённая программа не должна выполнять полезную часть.

Дополнительное задание: реализовать сравнение серийного номера, полученного из пароля, с эталоном в самомодифицирующемся коде и обойти эту проверку средствами реверс-инжиниринга и отладки.

2 ПРОГРАММА И ИСХОДНАЯ ПРОВЕРКА ПАРОЛЯ

2.1 Полезная нагрузка

Программа подсчитывает число вариантов расстановки знаков «+» и «−» между N числами, при которых получается заданная сумма. Первое число используется без изменения знака. Входной файл содержит число потоков, N, последовательность из N чисел и целевую сумму. Для перебора применяются рабочие потоки, критическая секция, стартовый семафор и код Грея. В output.txt записываются число потоков, N и число решений, в time.txt — время вычислений.

Листинг 1 — Входные данные input.txt

```
8
28
-7 -1 -5 6 -8 -3 1 3 4 -4 1 10 5 1 5 4 9 10 10 6 10 8 2 -6 7 8 1 -3 2
```

Для контрольного примера использованы 8 потоков и 28 чисел, целевая сумма равна 2. Контрольный результат — 3 220 095 решений. При правильном пароле secret123 в финальной защищённой версии формируется KEY0831623743. При wrong123 без патчинга выводится сообщение об ошибке, новые файлы результатов не создаются.

2.2 Проверка без защитных механизмов

В исходном учебном варианте пароль сравнивался с открытой строкой secret123 через strcmp. При совпадении записывался фиксированный номер KEY1234567890, выводилось сообщение об успехе и вызывалась функция programm.

Листинг 2 — Исходный вариант программы до добавления защит

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void programm()
{
    printf("Welcome to the program!\n");
    return;
}

int main()
{
    FILE* file = fopen("password.txt", "r");
    if (file == NULL)
```

```
    {
        printf("Error opening password file.\n");
        exit(1);
    }

    char password[100];
    if (fgets(password, sizeof(password), file) == NULL)
    {
        printf("Error reading password file.\n");
        fclose(file);
        exit(1);
    }

    int password_length = strlen(password);
    if (password[password_length - 1] == '\n')
    {
        password[password_length - 1] = '\0';
        password_length--;
    }

    printf("Password read from file: %s\n", password);

    const char* real_password = "secret123";
```

```
    if (strcmp(password, real_password) == 0)
    {
        FILE* out = fopen("serial.txt", "w");
        if (out == NULL)
        {
            printf("Error opening output file.\n");
            fclose(file);
            exit(1);
        }
        else
        {
            fprintf(out, "KEY$%010d$\n", 1234567890);
            fclose(out);
        }
        printf("Password is correct.\n");
        programm();
    }
    else
    {
        printf("Password is incorrect.\n");
    }

    fclose(file);
    return 0;
}
```

2.3 Первый бинарный патч

В x64dbg место проверки находилось через ссылки на строки secret123 и сообщения об успехе. После strcmp выполнялись test eax, eax и jne к ветке отказа. Поскольку strcmp возвращает 0 при совпадении, переход jne выполнялся при неверном пароле. Инструкция 75 70 занимала два байта и была заменена на 90 90, то есть два NOP. После этого выполнение продолжалось по ветке успеха независимо от результата сравнения. Этот эксперимент относится к раннему варианту программы с фиксированным номером; далее анализируется защищённая версия.

3 РЕАЛИЗАЦИЯ ЗАЩИТНЫХ МЕХАНИЗМОВ

3.1 Связь защит с состоянием программы

Большинство защит изменяет глобальную переменную g_seed через corrupt_seed. Исходное значение равно 0xDEADBEEF. От неё зависят ключ расшифровки сообщений, формирование серийного номера и целевая сумма в рабочих потоках. Поэтому обнаружение анализа может проявиться позже в виде нечитаемых сообщений, неверного номера или неправильного результата вычислений. IsDebuggerPresent обрабатывается отдельно: программа сразу завершается с кодом 1.

Листинг 3 — Формирование ключа и изменение состояния

```
// global seed for key generation
volatile unsigned int g_seed = 0xDEADBEEF;

static unsigned char str_key(void)
{
	return (unsigned char)((g_seed & 0xFF) ^ 0xB5);
}

static void corrupt_seed(void)
{
	g_seed = g_seed * 1664525u + 1013904223u;
}
//
```

3.2 Шифрование строк

Пароль и основные диагностические сообщения хранятся в массивах байтов, закодированных XOR с ключом 0x5A. Операция обратима: повторное применение того же ключа восстанавливает исходный байт. str_key получает 0x5A из младшего байта исходного g_seed: EF XOR B5 = 5A. При изменении состояния сообщения расшифровываются неправильно. Утилита xor_encrypt.cpp генерирует массивы для включения в основной код.

Листинг 4 — Расшифровка массива байтов

```
void decrypt(const unsigned char* enc, int len, unsigned char key, char* out)
{
    for (int i = 0; i < len; i++)
        out[i] = enc[i] ^ key;
    out[len] = '\0';
}
```

3.3 Обнаружение отладчика

В таблице 1 приведены методы обнаружения отладки.

Таблица 1 — Методы обнаружения отладки

Метод	Проверяемый признак	Реакция
IsDebuggerPresent	Наличие отладчика у текущего процесса	Сообщение и exit(1)
PEB.BeingDebugged	Байт по смещению 2 в PEB	corrupt_seed
CheckRemoteDebuggerPresent	Отладка текущего процесса	corrupt_seed
GetTickCount	Время короткого цикла больше 100 мс	corrupt_seed
Отладочные регистры	Ненулевые DR0–DR3	Попытка проверки через GetThreadContext

Листинг 5 — Реализация anti_debug_1

```
void anti_debug_1()
{
    if (IsDebuggerPresent())
    {
        print_enc(enc_dbg1, sizeof(enc_dbg1) - 1); printf("\n");
        exit(1);
    }
}
```

Листинг 6 — Реализация anti_debug_2

```
void anti_debug_2()
{
    PPEB peb = (PPEB)__readgsqword(0x60);
    if (peb->BeingDebugged)
    {
        corrupt_seed();
    }
}
```

Листинг 7 — Реализация anti_debug_3

```
void anti_debug_3()
{
    BOOL present = FALSE;
    CheckRemoteDebuggerPresent(GetCurrentProcess(), &present);
    if (present)
    {
        corrupt_seed();
    }
}
```

Листинг 8 — Реализация anti_debug_4

```
void anti_debug_4()
{
    DWORD start = GetTickCount();
    for (volatile int i = 0; i < 1000; i++) {}
    DWORD elapsed = GetTickCount() - start;
    if (elapsed > 100)
    {
        corrupt_seed();
    }
}
```

Листинг 9 — Реализация anti_debug_5

```
void anti_debug5()
{
	CONTEXT ctx;
	memset(&ctx, 0, sizeof(ctx));
	ctx.ContextFlags = CONTEXT_DEBUG_REGISTERS;
	if (GetThreadContext(GetCurrentThread(), &ctx))
	{
		if (ctx.Dr0 != 0 || ctx.Dr1 != 0 || ctx.Dr2 != 0 || ctx.Dr3 != 0)
			corrupt_seed();
	}
}
```

В Windows x64 адрес PEB считывается через GS:[0x60], затем проверяется BeingDebugged. CheckRemoteDebuggerPresent получает дескриптор текущего процесса. Таймингова проверка является эвристикой и может зависеть от планирования потоков.

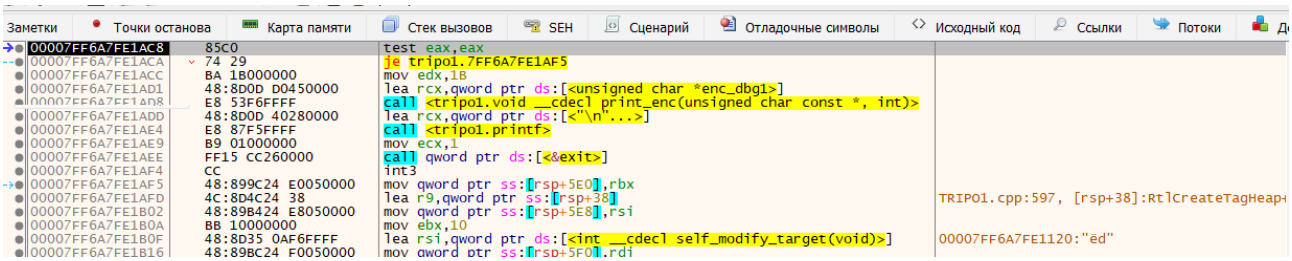


Рисунок 1 — Ветка завершения после обнаружения отладчика

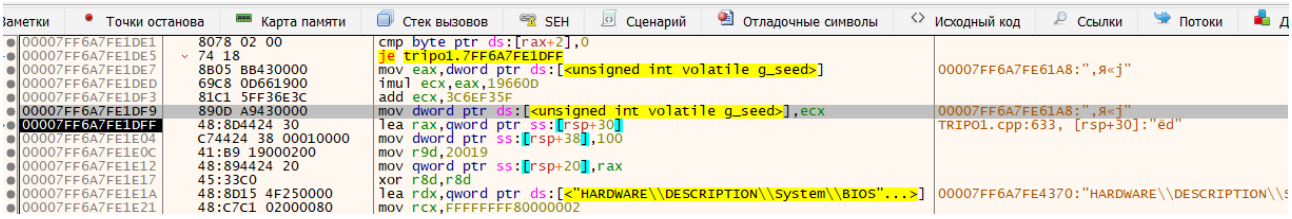


Рисунок 2 — Изменение g_seed после ручной проверки PEB

3.4 Выявление виртуальных машин

В anti_vm включены три способа: поиск характерного производителя системы в реестре BIOS, проверка префиксов MAC-адресов и поиск файлов драйверов VirtualBox и VMware. При обнаружении вызывается corrupt_seed. Проверка гипервизорного бита CPUID написана, но её вызов отключён и в число включённых методов не входит. Перечисленные признаки эвристические и могут отсутствовать в виртуальной среде либо сохраняться после её удаления.

В листинге 9 приведён код: Включённые проверки виртуальной среды.

Листинг 9 — Включённые проверки виртуальной среды

```
void anti_vm()
{
    if (check_registry_vm()) { corrupt_seed(); }
    if (check_mac_vm()) { corrupt_seed(); }
    //if (check_cpuid_vm()) { printf("VM: cpuid\n"); exit(7); }
    if (check_files_vm()) { corrupt_seed(); }
}
```

3.5 Контроль целостности и повторные проверки

verify_integrity вычисляет CRC32 байтов функции check_password_crc до функции-маркера marker_crc. Для сохранённого EXE начало имеет RVA 0x1930, маркер — 0x1970, длина участка — 64 байта. Независимый пересчёт дал 0xF5D709B8, совпадающий с эталоном в коде. При несовпадении меняется g_seed.

В листинге 10 приведён код: Проверка целостности функции пароля.

Листинг 10 — Проверка целостности функции пароля

```
void verify_integrity()
{
    unsigned char* start = (unsigned char*)check_password_crc;
    unsigned char* end = (unsigned char*)marker_crc;
    int len = (int)(end - start);
    if (len <= 0 || len > 4096) return;
    unsigned int current = crc32(start, len);
    if (current != 0xF5D709B8)
        corrupt_seed();
}
```

CRC32 не является криптографическим хешем и допускает коллизии. Расположение функций определяется компилятором и компоновщиком, поэтому границы и эталон привязаны к сборке. При некорректной длине текущая реализация пропускает проверку. В бинарном обходе изменялись вызывающие ветки, а не контролируемая функция, поэтому её CRC оставалась прежней.

Основная проверка выполняется в main, повторная — в programm через verify_password_again. Между ними распределены проверки среды и формирование номера. Ложная ветка strlen(password) == 999 расположена после чтения пароля; при буфере 100 байтов такое условие недостижимо и может быть устранено оптимизатором.

3.6 Противодействие анализу потока управления

В antidis_add1 после двух условных переходов расположен мусорный байт E8, который линейный дизассемблер может принять за начало CALL и включить последующие реальные инструкции в его операнд. В antidis_jmid безусловный переход перескакивает байт C7, который при разборе с неверной границы поглощает байты полезных инструкций. Процессор использует фактическую цель перехода, тогда как линейное отображение может оказаться неверным.

В листинге 11 приведён код: Два ASM приёма нарушения линейного разбора.

Листинг 11 — Два ASM приёма нарушения линейного разбора

```
.code


antidis_add1 PROC
    mov eax, ecx

    xor edx, edx
    jz  real_code
    jnz real_code

    db 0E8h

real_code:
    add eax, 1
    ret
antidis_add1 ENDP

antidis_jmid PROC
    mov eax, ecx


    db 0EBh, 001h  ;short jump+1

    db 0C7h


    add eax, 3
    ret
antidis_jmid ENDP
```

Дополнительно используется непрозрачный предикат (opaque_var | 1) != 0 и передача управления через SEH. Они усложняют анализ потока; отдельный третий метод, гарантированно приводящий к неправильному дизассемблированию, в данной версии не подтверждён.

В листинге 12 приведён код: Передача управления через исключение.

Листинг 12 — Передача управления через исключение

```
__declspec(noinline) void protected_action(const char* password)
{
    int zero = 0;
    zero = (opaque_var - opaque_var);

    __try
    {
        int crash = 1 / zero;
        printf("unreachable %d\n", crash);
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        programm(password);
    }
}
```

В protected_action намеренно вызывается целочисленное деление на ноль. Обработчик __except вызывает programm. В x64dbg наблюдалось first-chance исключение C0000094; команда Shift+F9 передавала его программе для обработки. Наличие обработчика видно в исходном коде и данных исключений PE, поэтому SEH не делает переход принципиально невидимым для анализа.

4 САМОМОДИФИЦИРУЮЩИЙСЯ КОД И ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ

4.1 Получение серийного номера из пароля

Серийный номер вычисляется алгоритмом FNV-1a с 32-битным беззнаковым накопителем. Каждый байт пароля смешивается XOR, затем результат умножается на 16777619 по модулю 2^32. Для secret123 получено 831623743, или 0x3191923F. Формат %010u дополняет десятичное значение нулём слева: KEY0831623743. FNV-1a используется для учебного отображения пароля в номер, а не для безопасного хранения паролей.

В листинге 13 приведён код: Вычисление серийного номера.

Листинг 13 — Вычисление серийного номера

```
unsigned int serial_from_password(const char* password)
{
    unsigned int hash = 2166136261u;

    for (const unsigned char* p = (const unsigned char*)password; *p != '\0'; ++p)
    {
        hash ^= *p;
        hash *= 16777619u;
    }

    return hash;
}
```

После хеширования номер смешивается с g_seed XOR DEADBEEF. При нормальном состоянии эта маска равна нулю. Затем вычитается 4, а ASM-функции прибавляют 1 и 3, сохраняя исходный результат и включая антидизассемблерные фрагменты в вычисление номера.

4.2 Изменение инструкции сравнения

В ASM-функции serial_check сравнивается ECX с непосредственной 32-битной константой. Начало команды cmp ecx, imm32 имеет байты 81 F9. Эталон 0x3191923F хранится в порядке little-endian как 3F 92 91 31. В EXE эти четыре байта предварительно закодированы XOR AA и равны 95 38 3B 9B. Метка serial_check_value указывает внутрь инструкции, на изменяемую константу.

В листинге 14 приведён код: Самомодифицируемая ASM функция сравнения.

Листинг 14 — Самомодифицируемая ASM функция сравнения

```
PUBLIC serial_check
PUBLIC serial_check_value

serial_check PROC
    ; cmp ecx, 3191923Fh
    ; Эталон внутри инструкции пока закодирован XOR AA.
    db 081h, 0F9h

serial_check_value LABEL BYTE
    db 095h, 038h, 03Bh, 09Bh

    sete al
    movzx eax, al
    ret
serial_check ENDP
```

В листинге 15 приведён код: Обёртка изменения и выполнения машинного кода.

Листинг 15 — Обёртка изменения и выполнения машинного кода

```
int check_serial_selfmod(unsigned int serial)
{
    DWORD old_protect;
    DWORD unused_protect;
    int matched;

    if (!VirtualProtect(serial_check_value, 4,
        PAGE_EXECUTE_READWRITE, &old_protect)) // virtualprotect позволяет изменять
инструкции
    {
        exit(20);
    }


    for (int i = 0; i < 4; ++i)
    {
        serial_check_value[i] ^= 0xAA;
    }

    if (!FlushInstructionCache(GetCurrentProcess(), // очистка кэша инструкций, чтобы
процессор видел изменения в коде
        serial_check_value, 4))
    {
        exit(21);
    }


    matched = serial_check(serial);


    for (int i = 0; i < 4; ++i)
    {
        serial_check_value[i] ^= 0xAA;
    }

    if (!FlushInstructionCache(GetCurrentProcess(),
        serial_check_value, 4))
    {
        exit(21);
    }


    if (!VirtualProtect(serial_check_value, 4,
```

```
        old_protect, &unused_protect))
    {
        exit(22);
    }

    return matched;
}
```

VirtualProtect временно разрешает запись, цикл XOR восстанавливает константу, FlushInstructionCache обеспечивает согласованность кэша инструкций. После вызова serial_check байты кодируются обратно и возвращаются прежние права доступа. Изменяется часть машинной инструкции, а не отдельная переменная с эталоном. Обёртка применяется последовательно из основного потока.

Результат наблюдения представлен на рисунке 3.



Рисунок 3 — Восстановленная инструкция сравнения с эталоном

4.3 Обход самомодифицирующейся проверки

В эксперименте на входе serial_check находился номер ECX = 0x8597F352, не совпадающий с эталоном 0x3191923F. После cmp флаг ZF был равен 0. До выполнения sete al флаг вручную изменён на 1. Затем sete al и movzx eax, al сформировали EAX = 1. Таким образом, функция сообщила об успехе при несовпадающем номере. Это обход в текущем процессе, не изменение файла на диске и не исправление самого номера.

Результат наблюдения представлен на рисунке 4.

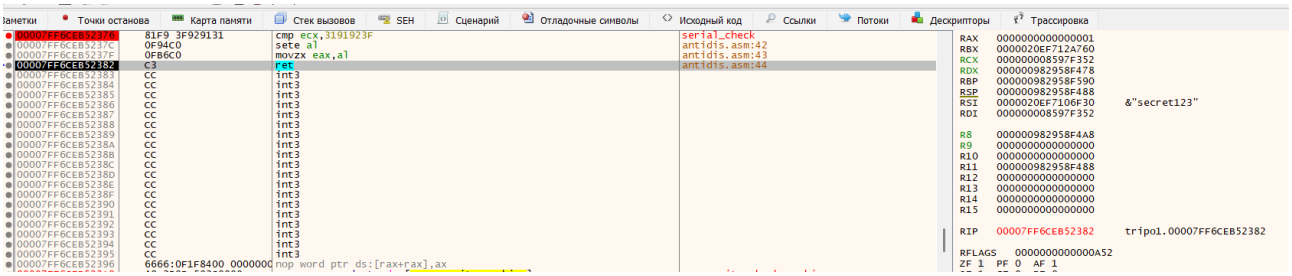


Рисунок 4 — Успешный возврат при несовпадающем номере после подмены ZF

После возврата обёртка повторно закодировала эталон. В дампе по адресу serial_check наблюдались 81 F9 95 38 3B 9B. Это подтверждает обе фазы изменения инструкции: восстановление для сравнения и обратное кодирование после него.

Результат наблюдения представлен на рисунке 5.

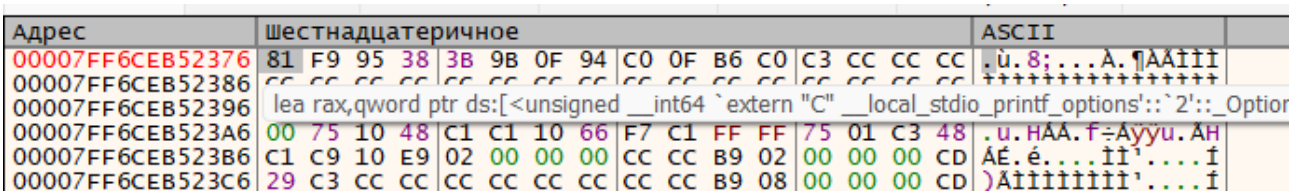


Рисунок 5 — Байты инструкции после обратного кодирования

5 БИНАРНЫЙ ОБХОД ЗАЩИЩЁННОЙ ПРОГРАММЫ

Правки выполнялись в x64dbg и сохранялись через окно исправлений в отдельные EXE. Изменения сохранялись до завершения процесса. Каждая следующая версия строилась на предыдущей, поэтому содержала накопленные правки. Программные точки использовались для наблюдения, аппаратные точки на запись — для первоначального поиска изменений g_seed.

5.1 Последовательность патчей

В таблице 2 приведены накопленные изменения машинного кода.

Таблица 2 — Накопленные изменения машинного кода

Версия	RVA	Изменение
patch1	1ACA	74 29 → EB 29; пропуск exit после IsDebuggerPresent
patch2	1DD5	74 18 → EB 18; пропуск порчи g_seed после PEB
patch3	20C3	74 18 → EB 18; пропуск порчи после CheckRemoteDebuggerPresent
patch4	209C–20A1	0F 85 2C 02 00 00 → шесть 90; пропуск отказа первой проверки
patch4	215A–2161	B9 3F 92 91 31 90 90 90; подстановка правильного хеша
patch4	17B5	75 29 → EB 29; пропуск отказа повторной проверки
patch5	1791	76 18 → EB 18; пропуск порчи состояния по таймингу

Пропуск проверок пароля сам по себе не давал правильного номера: FNV-1a от wrong123 отличается от хеша secret123. Поэтому две операции XOR при формировании номера заменены на mov ecx, 3191923F и три NOP. Следующие sub ecx, 4, antidis_add1 и antidis_jmid сохранены. Пять байтов MOV и три NOP занимают тот же восьмибайтовый участок.

Результат наблюдения представлен на рисунке 6.

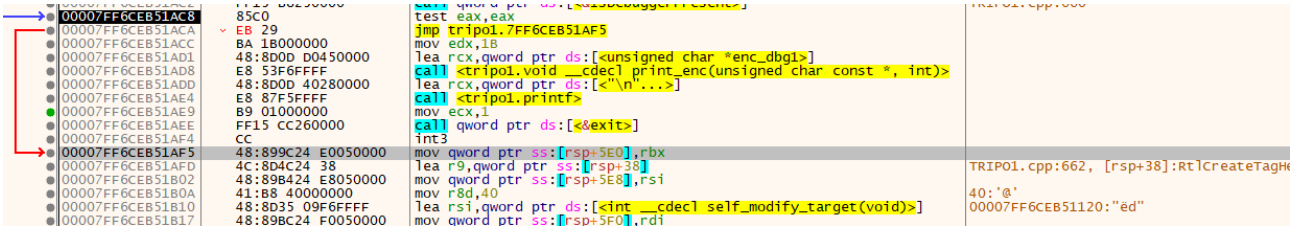


Рисунок 6 — Безусловный переход вместо реакции IsDebuggerPresent

Результат наблюдения представлен на рисунке 7.

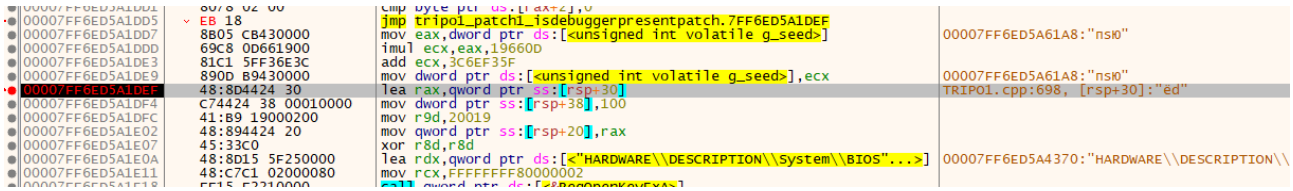


Рисунок 7 — Пропуск изменения состояния после проверки PEB

Результат наблюдения представлен на рисунке 8.

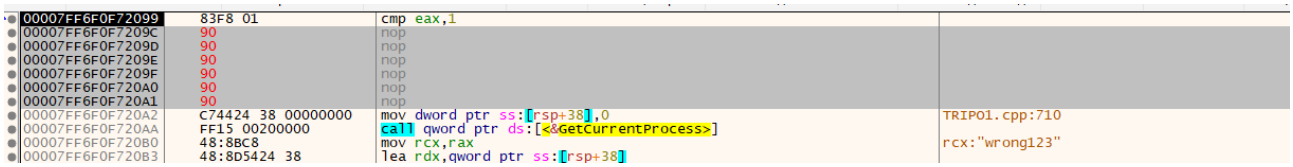


Рисунок 8 — Шесть NOP вместо перехода к отказу первой проверки

Результат наблюдения представлен на рисунке 9.

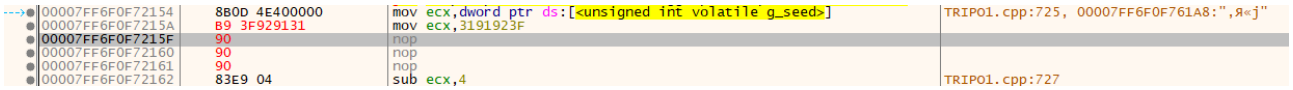


Рисунок 9 — Подстановка правильного значения для серийного номера

Результат наблюдения представлен на рисунке 10.

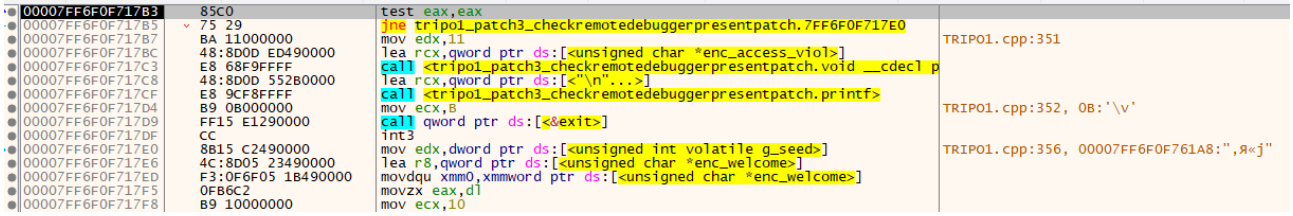


Рисунок 10 — Ветка отказа повторной проверки перед патчингом

Результат наблюдения представлен на рисунке 11.



Рисунок 11 — Таймингова проверка перед изменением состояния

5.2 Проверка результатов обхода

После первых трёх патчей запуск под отладчиком с правильным паролем завершался с кодом 0 и давал правильные результаты. Затем выполнялся обход с wrong123. Промежуточная версия давала правильный номер, но 0 решений: ветки пароля уже были пропущены, однако состояние полезных вычислений оставалось повреждённым. После патча таймингoвой ветки получены правильные результаты без ручного исправления регистров и памяти.

В таблице 3 приведены результаты контрольных запусков.

Таблица 3 — Результаты контрольных запусков

Сценарий	Серийный номер	Полезная часть
Исходная защищённая версия, secret123	KEY0831623743	3 220 095 решений
Исходная защищённая версия, wrong123	Не формируется	Не запускается
patch3, secret123, x64dbg	KEY0831623743	3 220 095 решений
patch5, wrong123, x64dbg	KEY0831623743	3 220 095 решений

При исследовании также обнаружена ошибка окружения: отсутствие input.txt приводило к exit(1), хотя защитные ветки уже были пройдены. После размещения входного файла вычисления запускались. Поэтому код выхода проверялся совместно с сообщениями и новыми файлами результатов; старые serial.txt и output.txt перед контрольными запусками удалялись.

6 УПАКОВКА И АНАЛИЗ ИСПОЛНЯЕМОГО ФАЙЛА

6.1 Упаковка с помощью UPX

Упакован исходный защищённый TRIPO1.exe, а не бинарный патч. Использована версия UPX 5.3.0 для Windows x64. Выходной файл записан отдельно, исходный сохранён. Вне отладчика с secret123 упакованная версия дала правильный серийный номер и контрольное число решений.

В листинге 16 приведён код: Команда и результат упаковки.

Листинг 16 — Команда и результат упаковки

```
upx.exe -o TRIPO1_upx.exe TRIPO1.exe
// Результат UPX: 18944 -> 11776, Ratio 62.16%, win64/pe
```

UPX хранит сжатое представление программы и код загрузчика. Во время запуска загрузчик распаковывает данные в память, подготавливает необходимые структуры исполнения и передаёт управление исходному коду. Упаковка не равнозначна шифрованию и не устраняет возможность динамического анализа.

6.2 Поиск оригинальной точки входа

В x64dbg точка входа упакованного файла находилась по адресу 00007FF6AA5EBEE0 при базе 00007FF6AA5E0000, то есть RVA BEE0. Здесь наблюдался код загрузчика. По известному RVA исходной точки входа 2650 установлена аппаратная точка на выполнение 00007FF6AA5E2650. После запуска она сработала на восстановленном коде mainCRTStartup.

Результат наблюдения представлен на рисунке 12.

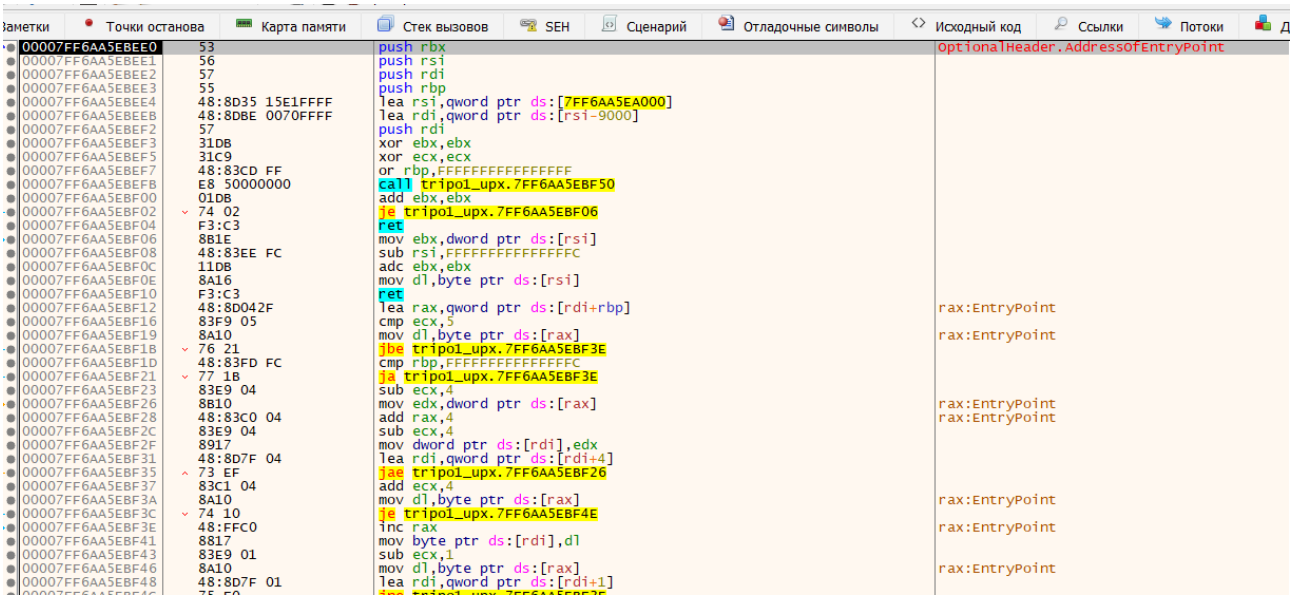


Рисунок 12 — Начало загрузчика UPX в точке входа

В листинге 17 приведён код: Фрагмент начала загрузчика по дизассемблеру.

Листинг 17 — Фрагмент начала загрузчика по дизассемблеру

```
push rbx
push rsi
push rdi
push rbp
lea rsi, [00007FF6AA5EA000h] ; адрес сжатых данных
lea rdi, [rsi-9000h]         ; адрес области восстановления
push rdi
xor ebx, ebx
xor ecx, ecx
or rbp, -1
call 00007FF6AA5EBF50h       ; подпрограмма загрузчика
```

Результат наблюдения представлен на рисунке 13.

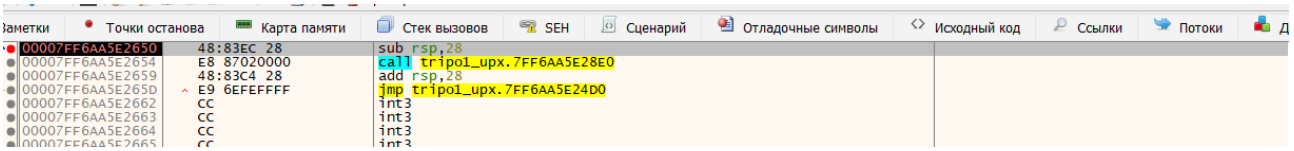


Рисунок 13 — Остановка на восстановленной OEP

Подтверждение выполнено сравнением первых машинных инструкций с исходным TRIPO1.exe: sub rsp, 28h; call __security_init_cookie; add rsp, 28h; jmp __scrt_common_main_seh. Байты совпали, а различие абсолютных адресов обусловлено базой загрузки. OEP относится к среде запуска C/C++, а не непосредственно к main.

Результат наблюдения представлен на рисунке 14.

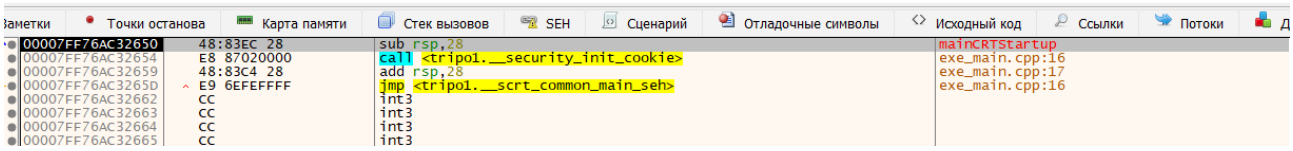


Рисунок 14 — Точка входа исходного EXE для сравнения

6.3 Размер энтропия и структура секций

Энтропия рассчитана по распределению всех байтов файла по формуле $H = -\Sigma\, p_i \log_2 p_i$, где p_i — частота значения байта. Диапазон показателя составляет от 0 до 8 бит/байт. Рост показателя после упаковки согласуется со сжатием, но сам по себе не доказывает шифрование.

В таблице 4 приведены параметры исходного и упакованного файлов.

Таблица 4 — Параметры исходного и упакованного файлов

Параметр	TRIPO1.exe	TRIPO1_upx.exe
Размер, байт	18 944	11 776
Энтропия, бит/байт	5,3786	7,0947
RVA точки входа	0x2650	0xBEE0

Упакованный файл занимает 62,16% исходного размера, уменьшение составляет 37,84%. Энтропия увеличилась на 1,7161 бит/байт. RawSize отражает данные секции в файле, VirtualSize — размер секции в памяти.

В таблице 5 приведены секции ре до и после упаковки в шестнадцатеричной записи.

Таблица 5 — Секции PE до и после упаковки в шестнадцатеричной записи

Файл и секция	RVA	VirtualSize	RawSize
Исходный .text	1000	2228	2400
Исходный .rdata	4000	15D8	1600
Исходный .data	6000	02F8	0200
Исходный .pdata	7000	021C	0400
Исходный .rsrc	8000	02B0	0400
Исходный .reloc	9000	002C	0200
UPX0	1000	9000	0000
UPX1	A000	3000	2400
Упакованный .rsrc	D000	1000	0600

В листинге 18 приведён код: Схема структуры упакованного файла.

Листинг 18 — Схема структуры упакованного файла

```
PE-заголовки: AddressOfEntryPoint = BEE0
  UPX0  RVA 1000, VirtualSize 9000, RawSize 0
        Область восстановления программы, OEP RVA 2650
  UPX1  RVA A000, VirtualSize 3000, RawSize 2400
        Сжатые данные и загрузчик, вход RVA BEE0
  .rsrc RVA D000, VirtualSize 1000, RawSize 0600
        Ресурсы
```

7 ВЫВОДЫ

В ходе работы исследованы бинарный патчинг и взаимосвязанные защитные механизмы. Реализованы проверки отладки и виртуальной среды, контроль целостности, повторные проверки пароля, XOR-кодирование выбранных строк, запутывание потока управления и самомодифицирующееся сравнение серийного номера. Показано, что подмена флага результата позволяет обойти сравнение даже после динамического восстановления инструкции.

Для защищённой версии получен сохранённый патч, который при неверном пароле формирует правильный серийный номер и сохраняет корректный результат полезных вычислений. Эксперимент показал необходимость контролировать общее состояние программы, а не только ветку отказа. Упаковка UPX уменьшила размер файла, повысила энтропию и изменила точку входа; исходная точка входа найдена после распаковки и подтверждена сравнением байтов.

Ограничения версии: шифрование строк неполное; отдельно не подтверждён третий метод некорректного дизассемблирования; проверка отладочных регистров требует доработки; границы CRC зависят от компоновки. Эти ограничения не отменяют полученные результаты, но не позволяют считать защиту полной или неуязвимой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Методические указания к лабораторной работе № 1 «Изучение методов защиты программного обеспечения».

Дополнительное задание к лабораторной работе № 1.

Sikorski M., Honig A. Practical Malware Analysis. No Starch Press, 2012.

Документация Microsoft Learn по IsDebuggerPresent, CheckRemoteDebuggerPresent, VirtualProtect, FlushInstructionCache и GetThreadContext. https://learn.microsoft.com/windows/win32/api/

UPX — Ultimate Packer for eXecutables. https://github.com/upx/upx

x64dbg. Документация отладчика. https://help.x64dbg.com/

ПРИЛОЖЕНИЕ 1 ИСХОДНЫЙ КОД ЗАЩИЩЁННОЙ ПРОГРАММЫ

Ниже приведён полный исходный текст сохранённой версии code/TRIPO1.cpp. Исходники при подготовке отчёта не изменялись.

В листинге 19 приведён код: Полный исходный текст TRIPO1.cpp.

Листинг 19 — Полный исходный текст TRIPO1.cpp

```
// нужно доделать шифрование всех строк


#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <winternl.h>
#include <iphlpapi.h>
#include <locale.h>
#pragma comment(lib, "iphlpapi.lib")
#include <intrin.h>

unsigned char enc_password[] = { 0x29, 0x3F, 0x39, 0x28, 0x3F, 0x2E, 0x6B, 0x68, 0x69, 0x5A };
unsigned char enc_success[] = { 0x0A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x33, 0x29,
0x7A, 0x39, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x74, 0x5A };
unsigned char enc_welcome[] = { 0x0D, 0x3F, 0x36, 0x39, 0x35, 0x37, 0x3F, 0x7A, 0x2E, 0x35, 0x7A,
0x2E, 0x32, 0x3F, 0x7A, 0x2A, 0x28, 0x35, 0x3D, 0x28, 0x3B, 0x37, 0x7B, 0x5A };
unsigned char enc_fail[] = { 0x0A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x33, 0x29, 0x7A,
 0x33, 0x34, 0x39, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x74, 0x5A };
unsigned char enc_err_openpwd[] = { 0x1F, 0x28, 0x28, 0x35, 0x28, 0x7A, 0x35, 0x2A, 0x3F, 0x34,
0x33, 0x34, 0x3D, 0x7A, 0x2A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x3C, 0x33, 0x36,
0x3F, 0x74, 0x5A };  // "Error opening password file."
unsigned char enc_err_readpwd[] = { 0x1F, 0x28, 0x28, 0x35, 0x28, 0x7A, 0x28, 0x3F, 0x3B, 0x3E,
0x33, 0x34, 0x3D, 0x7A, 0x2A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x3C, 0x33, 0x36,
0x3F, 0x74, 0x5A };  // "Error reading password file."
unsigned char enc_pwd_read[] = { 0x0A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x28, 0x3F,
0x3B, 0x3E, 0x7A, 0x3C, 0x28, 0x35, 0x37, 0x7A, 0x3C, 0x33, 0x36, 0x3F, 0x60, 0x7A, 0x7F, 0x29,
0x50, 0x5A };
unsigned char enc_dbg1[] = { 0x1E, 0x3F, 0x38, 0x2F, 0x3D, 0x3D, 0x3F, 0x28, 0x7A, 0x3E, 0x3F, 0x2E,
 0x3F, 0x39, 0x2E, 0x3F, 0x3E, 0x74, 0x7A, 0x1F, 0x22, 0x33, 0x2E, 0x33, 0x34, 0x3D, 0x74, 0x5A };
// "Debugger detected. Exiting."
unsigned char enc_fake[] = { 0x09, 0x3F, 0x39, 0x2F, 0x28, 0x33, 0x2E, 0x23, 0x7A, 0x2C, 0x33, 0x35,
 0x36, 0x3B, 0x2E, 0x33, 0x35, 0x34, 0x7A, 0x39, 0x35, 0x3E, 0x3F, 0x7A, 0x6A, 0x22, 0x6B, 0x1C,
0x74, 0x5A };  // "Security violation code 0x1F."
unsigned char enc_access_viol[] = { 0x1B, 0x39, 0x39, 0x3F, 0x29, 0x29, 0x7A, 0x2C, 0x33, 0x35,
0x36, 0x3B, 0x2E, 0x33, 0x35, 0x34, 0x74, 0x5A };  // "Access violation."
unsigned char enc_err_output[] = { 0x1F, 0x28, 0x28, 0x35, 0x28, 0x7A, 0x35, 0x2A, 0x3F, 0x34, 0x33,
 0x34, 0x3D, 0x7A, 0x35, 0x2F, 0x2E, 0x2A, 0x2F, 0x2E, 0x7A, 0x3C, 0x33, 0x36, 0x3F, 0x74, 0x5A };
// "Error opening output file."
unsigned char enc_correct[] = { 0x0A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x33, 0x29,
0x7A, 0x39, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x74, 0x5A };  // "Password is correct."
unsigned char enc_incorrect[] = { 0x0A, 0x3B, 0x29, 0x29, 0x2D, 0x35, 0x28, 0x3E, 0x7A, 0x33, 0x29,
0x7A, 0x33, 0x34, 0x39, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x74, 0x5A };  // "Password is
incorrect."
unsigned char enc_decoy[] = { 0x1B, 0x39, 0x39, 0x3F, 0x29, 0x29, 0x7A, 0x3E, 0x3F, 0x34, 0x33,
0x3F, 0x3E, 0x7A, 0x72, 0x3E, 0x3F, 0x39, 0x35, 0x23, 0x73, 0x74, 0x5A };  // "Access denied
(decoy)."
unsigned char enc_integrity[] = { 0x13, 0x34, 0x2E, 0x3F, 0x3D, 0x28, 0x33, 0x2E, 0x23, 0x7A, 0x39,
0x32, 0x3F, 0x39, 0x31, 0x7A, 0x3C, 0x3B, 0x33, 0x36, 0x3F, 0x3E, 0x74, 0x7A, 0x19, 0x35, 0x3E,
0x3F, 0x7A, 0x2E, 0x3B, 0x37, 0x2A, 0x3F, 0x28, 0x3F, 0x3E, 0x74, 0x5A };  // "Integrity check
failed. Code tampered."
unsigned char enc_finished[] = { 0x0A, 0x28, 0x35, 0x3D, 0x28, 0x3B, 0x37, 0x7A, 0x3C, 0x33, 0x34,
0x33, 0x29, 0x32, 0x3F, 0x3E, 0x74, 0x5A };  // "Program finished."
unsigned char enc_err_file[] = { 0x1F, 0x28, 0x28, 0x35, 0x28, 0x7A, 0x35, 0x2A, 0x3F, 0x34, 0x33,
0x34, 0x3D, 0x7A, 0x3C, 0x33, 0x36, 0x3F, 0x7B, 0x5A };  // "Error opening file!"


// global seed for key generation
volatile unsigned int g_seed = 0xDEADBEEF;

static unsigned char str_key(void)
{
```

```
    return (unsigned char)((g_seed & 0xFF) ^ 0xB5);
}

static void corrupt_seed(void)
{
    g_seed = g_seed * 1664525u + 1013904223u;
}
//


__declspec(noinline) int self_modify_target()
{
    int x = 50;
    x = x + 50;
    return x;
}


void self_modify_demo()
{
    unsigned char* func = (unsigned char*)self_modify_target;
    unsigned char* target_byte = func + 1;

    int (* volatile fp)() = self_modify_target;

    DWORD oldProtect;
    VirtualProtect(func, 16, PAGE_EXECUTE_READWRITE, &oldProtect);

    unsigned char key = 0xAA;
    *target_byte ^= key;
    FlushInstructionCache(GetCurrentProcess(), func, 16);



    *target_byte ^= key;
    FlushInstructionCache(GetCurrentProcess(), func, 16);

    VirtualProtect(func, 16, oldProtect, &oldProtect);


}



void anti_debug_4();
__declspec(noinline) int verify_password_again(const char* input);
extern "C" int antidis_add1(int x);
extern "C" int antidis_jmid(int x);
extern "C" int serial_check(unsigned int serial);
extern "C" unsigned char serial_check_value[];
void decrypt(const unsigned char* enc, int len, unsigned char key, char* out);

void print_enc(const unsigned char* enc, int len)
{
    char buf[256];
    decrypt(enc, len, str_key(), buf);
    printf("%s", buf);
}

//#include <intrin.h>
#define SUM_MAX (2147483647LL + 99999LL)
#define SUM_MIN (-2147483648LL - 99999LL)
#define LL long long
#define DEBUG_PRINT 0
#define SIZE 512
CRITICAL_SECTION crit_sec;
int count_threads = 0;
int* start_array = NULL;
int N = 0;
HANDLE start_sem;
LL result = 0;
DWORD WINAPI worker_thread(LPVOID param);
LL count_solved = 0;
LL next_task = 0;
DWORD start_time, end_time;
LL total = 0;
LL total_combinations();
```

```
int number_of_digits(LL task_index); // функция для номера знака (для кода Грэя)

FILE* open_file_and_scan()
{
    FILE* file = fopen("input.txt", "r");
    if (file == NULL)
    {
        print_enc(enc_err_file, sizeof(enc_err_file) - 1); printf("\n");
        exit(1);
    }
    return file;
}

void time_to_file()
{
    FILE* file = fopen("time.txt", "w");
    if (file == NULL)
    {
        print_enc(enc_err_file, sizeof(enc_err_file) - 1); printf("\n");
        exit(1);
    }
    DWORD elapsed = end_time - start_time;
    fprintf(file, "%lu", elapsed);
    fclose(file);
}

void result_to_file()
{
    FILE* file = fopen("output.txt", "w");
    if (file == NULL)
    {
        print_enc(enc_err_file, sizeof(enc_err_file) - 1); printf("\n");
        exit(1);
    }
    fprintf(file, "%d\n", count_threads);
    fprintf(file, "%d\n", N);
    fprintf(file, "%lld\n", count_solved);
    fclose(file);
}

int run_payload()
{
    setlocale(LC_ALL, "Russian");
    FILE* file = open_file_and_scan();
    fscanf(file, "%d", &count_threads);
    fscanf(file, "%d", &N);
    start_array = (int*)malloc(sizeof(int) * N);
    InitializeCriticalSection(&crit_sec);


    for (int i = 0; i < N; i++)
    {
        fscanf(file, "%d", &start_array[i]);
    }

    fscanf(file, "%lld", &result);
    fclose(file);


    total = total_combinations();


    start_sem = CreateSemaphore(NULL, 0, count_threads, NULL);
    HANDLE* threads = (HANDLE*)malloc(count_threads * sizeof(HANDLE));

    for (int i = 0; i < count_threads; i++)
    {
        threads[i] = CreateThread(NULL, 0, worker_thread, (LPVOID)(size_t)i, 0, NULL);
    }

    start_time = GetTickCount();


    ReleaseSemaphore(start_sem, count_threads, NULL);

    for (int i = 0; i < count_threads; i++)
    {
        WaitForSingleObject(threads[i], INFINITE);
```

```
    }

    end_time = GetTickCount();

    time_to_file();
    result_to_file();

    for (int i = 0; i < count_threads; i++)
        CloseHandle(threads[i]);

    free(threads);
    CloseHandle(start_sem);
    DeleteCriticalSection(&crit_sec);
    free(start_array);
    return 0;
}

LL total_combinations()
{
    return 1LL << (N - 1);
}

bool check_combination(LL task_index)
{
    int count_signs = N - 1;
    LL sum = (LL)start_array[0];

    for (int i = 0; i < count_signs; i++)
    {
        int digit = (task_index >> (count_signs - 1 - i)) & 1;
        if (digit == 1)
            sum += (LL)start_array[i + 1];
        else
            sum -= (LL)start_array[i + 1];

        if (sum > SUM_MAX || sum < SUM_MIN)
            return false;
    }

    if (sum == result)
    {
#if DEBUG_PRINT
        EnterCriticalSection(&crit_sec);
        printf("Решение: %d", start_array[0]);
        for (int i = 0; i < count_signs; i++)
        {
            int digit = (task_index >> (count_signs - 1 - i)) & 1;
            printf(" %c %d", digit ? '+' : '-', start_array[i + 1]);
        }
        printf(" = %lld\n", result);
        LeaveCriticalSection(&crit_sec);
#endif
        return true;
    }
    return false;
}

DWORD WINAPI worker_thread(LPVOID param)
{
    int idx = (int)(size_t)param;

    // Поток блокируется здесь, пока main не вызовет ReleaseSemaphore
    WaitForSingleObject(start_sem, INFINITE);


    LL poisoned_result = result ^ (LL)(g_seed ^ 0xDEADBEEF); // ЗАЩИТА

    while (true)
    {
        int index;
        int end_index;

        EnterCriticalSection(&crit_sec);
        if (next_task >= total)
        {
            LeaveCriticalSection(&crit_sec);
            break;
```

```
        }
        index = next_task;
        end_index = next_task + SIZE;
        if (end_index > total)
            end_index = (int)total;
        next_task = end_index;
        LeaveCriticalSection(&crit_sec);

        int local_solved = 0;
```

```
        int count_signs = N - 1; // количество знаков
        int current_digit_array[64] = { 0 }; // массив для хранения текущего знака каждого числа (0
- минус, 1 - плюс)
        LL summ = start_array[0]; // сумма текущей комбинации, начинаем с первого числа (тк оно
всегда положительное)
        int Grey_code = index ^ (index >> 1); // код Грея для текущего индекса (xor - ^)
        for (int k = 0; k < count_signs; k++)
        {
            int Grey_code_new = Grey_code >> k;
            Grey_code_new = Grey_code_new & 1; // получение знака
            current_digit_array[k] = Grey_code_new; // запись текущего знака (для текущего числа) в
массив
            if (Grey_code_new == 1) // обработка знака
            {
                summ += start_array[k + 1];
            }
            else
            {
                summ -= start_array[k + 1];
            }
        }

        if (summ == poisoned_result) // ЗАЩИТА
        {
            local_solved++;
        }
        for (int i = index + 1; i < end_index;i++)
        {
            int k = number_of_digits(i); // получили позицию знака, который изменился
            current_digit_array[k] = 1 - current_digit_array[k]; // меняем знак на противоположный
            if (current_digit_array[k] == 1) // обработка знака
            {
                summ += 2 * start_array[k + 1]; // прибавляем дважды, тк мы меняем знак с минуса на
плюс
            }
            else
            {
```

```
                summ -= 2 * start_array[k + 1]; // вычитаем дважды, тк мы меняем знак с плюса на
минус
            }
            if (summ == poisoned_result)
            {
                local_solved++;
            }
        }
        if (local_solved > 0)
        {
            EnterCriticalSection(&crit_sec);
            count_solved += local_solved;
            LeaveCriticalSection(&crit_sec);
        }
    }

    return 0;
}

int number_of_digits(LL task_index) // суть в том чтобы найти позицию первой еденицы справа
{
    int c = 0;
    while ((task_index & 1) == 0)
    {
        task_index >>= 1;
        c++;
    }
    return c;
}
```

```
void programm(const char* password)
{
    anti_debug_4();
    if (!verify_password_again(password))
    {
        print_enc(enc_access_viol, sizeof(enc_access_viol) - 1); printf("\n");
        exit(11);
    }

    char buf[64];
    decrypt(enc_welcome, sizeof(enc_welcome) - 1, str_key(), buf);
    printf("%s\n", buf);
    run_payload();


    return;
}

void decrypt(const unsigned char* enc, int len, unsigned char key, char* out)
{
    for (int i = 0; i < len; i++)
        out[i] = enc[i] ^ key;
    out[len] = '\0';
}

void anti_debug_1()
{
    if (IsDebuggerPresent())
    {
        print_enc(enc_dbg1, sizeof(enc_dbg1) - 1); printf("\n");
        exit(1);
    }
}


void anti_debug_2()
{
    PPEB peb = (PPEB)__readgsqword(0x60);
    if (peb->BeingDebugged)
    {
        corrupt_seed();
    }
}


void anti_debug_3()
{
    BOOL present = FALSE;
    CheckRemoteDebuggerPresent(GetCurrentProcess(), &present);
    if (present)
    {
        corrupt_seed();
    }
}


void anti_debug_4()
{
    DWORD start = GetTickCount();
    for (volatile int i = 0; i < 1000; i++) {}
    DWORD elapsed = GetTickCount() - start;
    if (elapsed > 100)
    {
        corrupt_seed();
    }
}


void anti_debug5()
{
    CONTEXT ctx;
    memset(&ctx, 0, sizeof(ctx));
    ctx.ContextFlags = CONTEXT_DEBUG_REGISTERS;
```

```
    if (GetThreadContext(GetCurrentThread(), &ctx))
    {
        if (ctx.Dr0 != 0 || ctx.Dr1 != 0 || ctx.Dr2 != 0 || ctx.Dr3 != 0)
            corrupt_seed();
    }
}
```

```
int check_registry_vm()
{
    HKEY hKey;
    char buffer[256];
    DWORD size = sizeof(buffer);


    if (RegOpenKeyExA(HKEY_LOCAL_MACHINE,
        "HARDWARE\\DESCRIPTION\\System\\BIOS", 0, KEY_READ, &hKey) == ERROR_SUCCESS)
    {
        if (RegQueryValueExA(hKey, "SystemManufacturer", NULL, NULL,
            (LPBYTE)buffer, &size) == ERROR_SUCCESS)
        {
            RegCloseKey(hKey);
            if (strstr(buffer, "VMware") || strstr(buffer, "VirtualBox") ||
                strstr(buffer, "VBOX") || strstr(buffer, "innotek"))
                return 1;
        }
        else RegCloseKey(hKey);
    }
    return 0;
}


int check_mac_vm()
{
    IP_ADAPTER_INFO adapters[16];
    DWORD bufLen = sizeof(adapters);
    if (GetAdaptersInfo(adapters, &bufLen) != ERROR_SUCCESS) return 0;

    IP_ADAPTER_INFO* a = adapters;
    while (a)
    {
        // VirtualBox: 08-00-27
        if (a->Address[0] == 0x08 && a->Address[1] == 0x00 && a->Address[2] == 0x27)
            return 1;
        // VMware: 00-0C-29
        if (a->Address[0] == 0x00 && a->Address[1] == 0x0C && a->Address[2] == 0x29)
            return 1;
        a = a->Next;
    }
    return 0;
}


int check_cpuid_vm()
{
    int cpuInfo[4] = { 0 };
    __cpuid(cpuInfo, 1);
    if ((cpuInfo[2] >> 31) & 1)
        return 1;
    return 0;
}

int check_files_vm()
{
    const char* vm_files[] = {
        "C:\\Windows\\System32\\drivers\\VBoxMouse.sys",
        "C:\\Windows\\System32\\drivers\\VBoxGuest.sys",
        "C:\\Windows\\System32\\drivers\\VBoxSF.sys",
        "C:\\Windows\\System32\\drivers\\vmhgfs.sys",
        "C:\\Windows\\System32\\drivers\\vmmouse.sys"
    };
    int count = sizeof(vm_files) / sizeof(vm_files[0]);
    for (int i = 0; i < count; i++)
    {
        if (GetFileAttributesA(vm_files[i]) != INVALID_FILE_ATTRIBUTES)
            return 1;
    }
```

```
    return 0;
}

void anti_vm()
{
    if (check_registry_vm()) { corrupt_seed(); }
    if (check_mac_vm()) { corrupt_seed(); }
    //if (check_cpuid_vm()) { printf("VM: cpuid\n"); exit(7); }
    if (check_files_vm()) { corrupt_seed(); }
}



unsigned int crc32(unsigned char* data, int len)
{
    unsigned int crc = 0xFFFFFFFF; // стартовое значение
    for (int i = 0; i < len; i++)
    {
        crc ^= data[i];
        for (int j = 0; j < 8; j++)
            crc = (crc >> 1) ^ (0xEDB88320 & (-(int)(crc & 1))); // полином
    }
    return ~crc;
}

volatile int opaque_var = 5;




__declspec(noinline) int check_password_crc(const char* input, const char* real)
{
    int i = 0;

    while (real[i] != '\0')
    {
        if (input[i] != real[i])
            return 0;
        i++;
    }
    if (input[i] != '\0')
        return 0;
    return 1;
}
```

```
__declspec(noinline) void marker_crc()
{
    volatile int x = 0;
    (void)x;
}

void verify_integrity()
{
    unsigned char* start = (unsigned char*)check_password_crc;
    unsigned char* end = (unsigned char*)marker_crc;
    int len = (int)(end - start);
    if (len <= 0 || len > 4096) return;
    unsigned int current = crc32(start, len);
    if (current != 0xF5D709B8)
        corrupt_seed();
}


__declspec(noinline) int verify_password_again(const char* input)
{
    char real[64];
    decrypt(enc_password, sizeof(enc_password) - 1, str_key(), real);


    int i = 0;
    while (real[i] != '\0')
    {
        if (input[i] != real[i])
            return 0;
        i++;
    }
```

```
    if (input[i] != '\0') return 0;
    return 1;
}
```

```
__declspec(noinline) void protected_action(const char* password)
{
    int zero = 0;
    zero = (opaque_var - opaque_var);

    __try
    {
        int crash = 1 / zero;
        printf("unreachable %d\n", crash);
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        programm(password);
    }
}


unsigned int serial_from_password(const char* password)
{
    unsigned int hash = 2166136261u;

    for (const unsigned char* p = (const unsigned char*)password; *p != '\0'; ++p)
    {
        hash ^= *p;
        hash *= 16777619u;
    }

    return hash;
}


int check_serial_selfmod(unsigned int serial)
{
    DWORD old_protect;
    DWORD unused_protect;
    int matched;

    if (!VirtualProtect(serial_check_value, 4,
        PAGE_EXECUTE_READWRITE, &old_protect)) // virtualprotect позволяет изменять инструкции
    {
        exit(20);
    }


    for (int i = 0; i < 4; ++i)
    {
        serial_check_value[i] ^= 0xAA;
    }

    if (!FlushInstructionCache(GetCurrentProcess(), // очистка кэша инструкций, чтобы процессор
видел изменения в коде
        serial_check_value, 4))
    {
        exit(21);
    }


    matched = serial_check(serial);


    for (int i = 0; i < 4; ++i)
    {
        serial_check_value[i] ^= 0xAA;
    }

    if (!FlushInstructionCache(GetCurrentProcess(),
        serial_check_value, 4))
    {
        exit(21);
    }
```

```
    if (!VirtualProtect(serial_check_value, 4,
        old_protect, &unused_protect))
    {
        exit(22);
    }

    return matched;
}

int main()
{
    anti_debug_1();

    self_modify_demo();

    verify_integrity();
    FILE* file = fopen("password.txt", "r");
    if (file == NULL)
    {
        print_enc(enc_err_openpwd, sizeof(enc_err_openpwd) - 1); printf("\n");
        exit(1);
    }

    char password[100];
    if (fgets(password, sizeof(password), file) == NULL)
    {
        print_enc(enc_err_readpwd, sizeof(enc_err_readpwd) - 1); printf("\n");
        fclose(file);
        exit(1);
    }

    int password_length = strlen(password);
    if (password[password_length - 1] == '\n')
    {
        password[password_length - 1] = '\0';
        password_length--;
    }

    { char fb[256]; decrypt(enc_pwd_read, sizeof(enc_pwd_read) - 1, str_key(), fb); printf(fb,
password); }
```

```
    if (strlen(password) == 999)   // fake
    {
        print_enc(enc_fake, sizeof(enc_fake) - 1); printf("\n");
        exit(42);
    }

    anti_debug_2();

    anti_vm();

    anti_debug5();

    char real_password[64];
    decrypt(enc_password, sizeof(enc_password) - 1, str_key(), real_password);



    if (check_password_crc(password, real_password) == 1)
    {

        anti_debug_3();


        FILE* out = fopen("serial.txt", "w");
        if (out == NULL)
        {
            print_enc(enc_err_output, sizeof(enc_err_output) - 1); printf("\n");
            fclose(file);
            exit(1);
        }
        else
        {

            unsigned int serial_num = serial_from_password(password);
```

```
            serial_num ^= (g_seed ^ 0xDEADBEEFu);

            serial_num -= 4u;
            serial_num = (unsigned int)antidis_add1((int)serial_num);
            serial_num = (unsigned int)antidis_jmid((int)serial_num);

            if (!check_serial_selfmod(serial_num))
            {
                fclose(out);
                fclose(file);
                exit(12);
            }

            fprintf(out, "KEY$%010u$\n", serial_num);
            fclose(out);
        }

        if ((opaque_var | 1) != 0) // непрозрачный предикат
        {
            print_enc(enc_correct, sizeof(enc_correct) - 1); printf("\n");
            protected_action(password);
        }
        else
        {
            print_enc(enc_decoy, sizeof(enc_decoy) - 1); printf("\n");
            exit(99);
        }
    }
    else
    {
        print_enc(enc_incorrect, sizeof(enc_incorrect) - 1); printf("\n");
    }
    print_enc(enc_finished, sizeof(enc_finished) - 1); printf("\n");
    fclose(file);
    return 0;
}
```

ПРИЛОЖЕНИЕ 2 АССЕМБЛЕРНЫЕ ФУНКЦИИ И УТИЛИТА КОДИРОВАНИЯ

В листинге 20 приведён код: Полный исходный текст antidis.asm.

Листинг 20 — Полный исходный текст antidis.asm

```
.code


antidis_add1 PROC
    mov eax, ecx

    xor edx, edx
    jz  real_code
    jnz real_code

    db 0E8h

real_code:
    add eax, 1
    ret
antidis_add1 ENDP

antidis_jmid PROC
    mov eax, ecx


    db 0EBh, 001h  ;short jump+1

    db 0C7h


    add eax, 3
    ret
antidis_jmid ENDP

PUBLIC serial_check
PUBLIC serial_check_value

serial_check PROC
    ; cmp ecx, 3191923Fh
    ; Эталон внутри инструкции пока закодирован XOR AA.
    db 081h, 0F9h
```

```
serial_check_value LABEL BYTE
    db 095h, 038h, 03Bh, 09Bh

    sete al
    movzx eax, al
    ret
serial_check ENDP

END
```

В листинге 21 приведён код: Полный исходный текст xor_encrypt.cpp.

Листинг 21 — Полный исходный текст xor_encrypt.cpp

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>

void emit(const char* name, const char* text, unsigned char key)
{
    size_t len = strlen(text);
    printf("unsigned char %s[] = { ", name);
    for (size_t i = 0; i < len; i++)
        printf("0x%02X, ", (unsigned char)text[i] ^ key);
    printf("0x%02X };  // \"%s\"\n", key, text);
}
```

```
int main()
{
    unsigned char key = 0x5A;

    emit("enc_err_openpwd", "Error opening password file.", key);
    emit("enc_err_readpwd", "Error reading password file.", key);
    emit("enc_pwd_read", "Password read from file: %s\n", key);
    emit("enc_dbg1", "Debugger detected. Exiting.", key);
    emit("enc_fake", "Security violation code 0x1F.", key);
    emit("enc_access_viol", "Access violation.", key);
    emit("enc_err_output", "Error opening output file.", key);
    emit("enc_correct", "Password is correct.", key);
    emit("enc_incorrect", "Password is incorrect.", key);
    emit("enc_decoy", "Access denied (decoy).", key);
    emit("enc_integrity", "Integrity check failed. Code tampered.", key);
    emit("enc_finished", "Program finished.", key);
    emit("enc_err_file", "Error opening file!", key);
    return 0;
}
```

ПРИЛОЖЕНИЕ 3 КОНТРОЛЬНЫЕ СУММЫ ИСПОЛНЯЕМЫХ ФАЙЛОВ

SHA-256 идентифицирует конкретные файлы, использованные при подготовке отчёта. Контрольные суммы позволяют отличать сохранённые варианты после пересборки или повторного патчинга.

```
TRIPO1.exe
6eebeafdc4972645cf3d28150a7a08394196314d147c2a83b5a3c050bd4cfdd6
TRIPO1_upx.exe
61e78ae8e509929ef3a2a5defd1f31241976b428f3ed34b016c0e50d90246281
xor_encrypt.exe
ceb182c5cc7dd9b0d8ca5f208aff9557b6d34b4159c7b480bf649054e7cb64f2
TRIPO1_patch1_IsDebuggerPresentPatch.exe
fee24c721452fdfcbe5db98dd280648e1b550a38d7b6abbc7ec853bd1d845b7f
TRIPO1_patch2_PebPatch.exe
092c56e88f39b61f77073270812345087l662b3964116c73d7802ecc66fb4487
TRIPO1_patch3_CheckRemoteDebuggerPresentPatch.exe
82cb2183c59150d63092e3990b38175b5ce7fa79a39cc031e09a8dc90aa0529b
TRIPO1_patch4_PasswordByPassPatch.exe
bbec57f2bbe2e7a2903dd1d19a8cb9182e49f725dbdaaefdf9328809b266a987
TRIPO1_patch5_TimingPatch.exe
b98bb245aa50ddd4cc54c0569bad611f42c05b793d17bddf9d814e2019915599
```